Labyrinth Documentation

1 Labyrinth as a CSP

- Goal: Given a parameter size n and a minimum number of walls $nWalls$ (excluding the border), find a solvable labyrinth constellation with an entry and an exit.

Problem Formulation as CSP:

$$CSP = \langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$$

1.0.1 Variables $\mathcal{X}$

For a $n \times n$ grid we create $n \times n$ variables, each denoting one cell in the labyrinth grid:

$$\mathcal{X} = \{X_0, X_1, X_2, \ldots, X_{n \times n-1}\}$$

1.0.2 Domains $\mathcal{D}$

We denote a wall in the labyrinth as 1 and an empty cell as 0. Every variable can either be a wall or an empty cell:

$$\mathcal{D} = \{D_0, D_1, D_2, \ldots, D_{n \times n-1}\}$$

where

$$D_0 = D_1 = D_2 = \cdots = D_{n \times n-1} = \{0, 1\}$$

1.0.3 Constraints $\mathcal{C}$

Each Constraint can be defined as:

$$C = \begin{cases} Scope(C) & \subseteq \mathcal{X} \\ Relation(C) & \subseteq \prod_{x \in Sc(C)} D_x \end{cases}$$

We differentiate between Tabular Constraints and Predicate Constraints.

Tabular Constraints The Relation, as per definition, is expressed as a set of allowed tuples for the constraint.

Predicate Constraints The Relation is expressed as a function.

Furthermore, constraints can be differentiated based on their *arity*. The arity of a constraint is the number of variables involved in the constraint i.e. the cardinality of its scope (Unary Constraints, Binary Constraints, ..., n-ary Constraints). For CSP-solvers it is desirable to keep the arity small. Any n-ary CSP can be converted into a binary constraint.

For the labyrinth finder we define the following constraints:

Constraint 1: The top and bottom border of the labyrinth are all walls Instead of expressing a single $2n$-ary constraint involving all top and bottom border variables, we can express $2n$ unary constraints.

First we find all necessary border variables:

$$\mathcal{X}_{TBborder} = \{X_{B,0}, X_{B,1}, \ldots, X_{B,2n}\} \subseteq \mathcal{X}$$

For each Border variable $X_{B,i}$ we define the following constraint:

$$C_{1,i} = \begin{cases} Scope(C_{1,i}) = \{X_{B,i}\} \\ Relation(C_{1,i}) = \{(1)\} \end{cases}$$

- Unary Constraint
- Tabular Constraint

Constraint 2: The left border is all walls except exactly one opening (start). Once again we define all left border variables as follows:

$$\mathcal{X}_{leftborder} = \{X_{B,0}, X_{B,1}, \ldots, X_{B,2n}\} \subseteq \mathcal{X}$$

Due to the dependence within the variables, we will define a single $2n$-ary constraint.

$$C_2 = \begin{cases} Scope(C_2) = \{X_{B,0}, X_{B,1}, \ldots, X_{B,2n}\} \\ Relation(C_2) = \{(0,1,1,...), (1,0,1,...), (1,1,0,...), \ldots\} \end{cases}$$

For the relation, we have exactly n possibilities to distribute one opening among n cells. More precisely this can be expressed as follows:

$$Relation(C_2) = \{(x_1, \ldots, x_n) \mid \exists j \in \{1, \ldots, n\} : \ x_j = k, \ x_i = 0 \text{ for all } i \neq j\}$$

- $2n$-ary Constraint
- Tabular Constraint

Constraint 3: The right border is all wall except exactly one opening (goal). We define this constraint equivalently to constraint 2:

$$\mathcal{X}_{rightborder} = \{X_{B,0}, X_{B,1}, \ldots, X_{B,2n}\} \subseteq \mathcal{X}$$

$$C_3 = \begin{cases} Scope(C_2) = \{X_{B,0}, X_{B,1}, \ldots, X_{B,2n}\} \\ Relation(C_2) = \{(0,1,1,...), (1,0,1,...), (1,1,0,...), \ldots\} \end{cases}$$

or alternatively:

$$Relation(C_3) = \{(x_1, \ldots, x_n) \mid \exists j \in \{1, \ldots, n\} : \ x_j = k, \ x_i = 0 \text{ for all } i \neq j\}$$

- $2n$-ary Constraint
- Tabular Constraint

Constraint ?: All open non-border cells have to be reachable from the entrance.

Constraint ?: Every 2x2 block has to contain at least one open cell.

2 Implementation

2.1 CSP Solver

Backtracking search explores partial assignments recursively and returns when it finds a complete assignment satisfying all constraints.

2.2 Backtracking search pseudocode

Algorithm 1 Backtracking search for a CSP

```
1: function BACKTRACKING-SEARCH(csp)
2:     return BACKTRACK(csp, ∅)
3: end function

4: function BACKTRACK(csp, assignment)
5:     if assignment is complete then
6:         return assignment
7:     end if
8:     var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
9:     for all value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
10:        if value is consistent with assignment then
11:            add {var = value} to assignment
12:            inferences ← INFERENCE(csp, var, assignment)
13:            if inferences ≠ failure then
14:                add inferences to csp
15:                result ← BACKTRACK(csp, assignment)
16:                if result ≠ failure then
17:                    return result
18:                end if
19:                remove inferences from csp
20:            end if
21:            remove {var = value} from assignment
22:        end if
23:    end for
24:    return failure
25: end function
```

Variable selection can use the degree heuristic, domain values can be ordered by the least-constraining-value heuristic, and inference can use forward checking, as intended in the project's CSP solver.

3 Search algorithms to solve a CSP